System Verification and Validation Plan for Software Engineering

Team 19, Team BVSH
Shivya Mehta
Anindita Bornomala
Harmanpreet Singh Sagar
Vanessa Lai

November 3, 2026

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 2
- 2.4 Relevant Documentation . . . 2

3 Plan **3**
- 3.1 Verification and Validation Team . . . 3
- 3.2 SRS Verification . . . 3
- 3.3 Design Verification . . . 3
- 3.4 Verification and Validation Plan Verification . . . 3
- 3.5 Implementation Verification . . . 4
- 3.6 Automated Testing and Verification Tools . . . 4
- 3.7 Software Validation . . . 4

4 System Tests **5**
- 4.1 Tests for Functional Requirements . . . 5
  - 4.1.1 Area of Testing1 . . . 5
  - 4.1.2 Area of Testing2 . . . 6
- 4.2 Tests for Nonfunctional Requirements . . . 6
  - 4.2.1 Course Outline Processing . . . 6
  - 4.2.2 Calendar and Web Application . . . 8
  - 4.2.3 Security, Reliability, and Deployment . . . 10
- 4.3 Traceability Between Test Cases and Requirements . . . 12

5 Unit Test Description **12**
- 5.1 Unit Testing Scope . . . 12
- 5.2 Tests for Functional Requirements . . . 12
  - 5.2.1 Module 1 . . . 12
  - 5.2.2 Module 2 . . . 13
- 5.3 Tests for Nonfunctional Requirements . . . 14
  - 5.3.1 Module ? . . . 14
  - 5.3.2 Module ? . . . 14
- 5.4 Traceability Between Test Cases and Modules . . . 14

6 Appendix **16**
6.1 Symbolic Parameters 16
6.2 Usability Survey Questions? 16

7 Appendix — Generative AI Use Disclosure **17**
7.1 AI Tools Used 17
7.2 How and Where Used 17
7.3 Verification of AI-Generated Content 17

8 Appendix — Reflection **18**

List of Tables

[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
ASSURE	Accreditation and Scheduling Support for Undergraduate Review and Evaluation
CAS	Department of Computing and Software
CEAB	Canadian Engineering Accreditation Board, the organization responsible for accrediting undergraduate engineering programs in Canada.
LLM	Large Language Model
PDF	Portable Document Format
SRS	Software Requirements Specification (Mehta et al., 2026)
V&V	Verification and Validation

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

2.1 Summary

The software under test is ASSURE (Accreditation and Scheduling Support for Undergraduate Review and Evaluation), which is a course-outline review system for the Department of Computing and Software (CAS) at McMaster University.

ASSURE accepts a batch of course outlines in PDF (Word and LaTeX document support will be a future feature). A replaceable (black box) locally hosted open-weight large language model (LLM) extracts each assessment's name, date, weight, and learning outcomes, and it either cites the passage each value came from or reports that it cannot find the value. The system checks the extracted information against McMaster University's Undergraduate Course Management Policy (McMaster University Senate, 2025), academic-calendar restrictions, and the Canadian Engineering Accreditation Board's (CEAB) Accreditation Criteria and Procedures (Canadian Engineering Accreditation Board, 2025), and flags missing, incomplete, or non-compliant information for review. Its dashboard supports filtering by course, year, and program. It also displays term completeness, workload or scheduling conflicts, and allows users to override or correct extracted values. The system also stores the reviewed data for export, while its shared calendar presents assessment deadlines across CAS courses and highlights items that need attention.

2.2 Objectives

[State what is intended to be accomplished. The objective will be around the qualities that are most important for your project. You might have something like: "build confidence in the software correctness," "demonstrate adequate usability." etc. You won't list all of the qualities, just those that are most important. —SS]

[You should also list the objectives that are out of scope. You don't have the resources to do everything, so what will you be leaving out. For instance,

if you are not going to verify the quality of usability, state this. It is also worthwhile to justify why the objectives are left out. —SS]

[The objectives are important because they highlight that you are aware of limitations in your resources for verification and validation. You can't do everything, so what are you going to prioritize? As an example, if your system depends on an external library, you can explicitly state that you will assume that external library has already been verified by its implementation team. —SS]

2.3 Custom Documents

The project includes two approved Custom Documents that support the design process and end-user adoption:

1. **Fall Term CD: Wireframe Report.** The wireframe report will document the proposed administrative dashboard, including the shared assessment calendar, course filtering tools, and scheduling recommendation displays. The wireframes will be reviewed with the project clients and stakeholders to confirm that the navigation, component interactions, and workflows reflect their needs before front-end implementation begins. Feedback from this review will be used to revise the interface design.

2. **Winter Term CD: User Manual.** The user manual will provide instructors and CAS administrative staff with step-by-step instructions for uploading course outlines, filtering the centralized assessment calendar, exporting database records, and reviewing policy compliance flags and scheduling recommendations. The manual will be checked against the implemented system and reviewed for completeness, clarity, and whether users can follow its procedures to complete the documented tasks.

2.4 Relevant Documentation

[Reference relevant documentation. This will definitely include your SRS and your other project documents (design documents, like MG, MIS, etc). You can include these even before they are written, since by the time the project is done, they will be written. You can create BibTeX entries for your

documents and within those entries include a hyperlink to the documents. —SS]

Author (2019)

[Don't just list the other documents. You should explain why they are relevant and how they relate to your VnV efforts. —SS]

3 Plan

[Introduce this section. You can provide a roadmap of the sections to come. —SS]

3.1 Verification and Validation Team

[Your teammates. Maybe your supervisor. You should do more than list names. You should say what each person's role is for the project's verification. A table is a good way to summarize this information. —SS]

3.2 SRS Verification

[List any approaches you intend to use for SRS verification. This may include ad hoc feedback from reviewers, like your classmates (like your primary reviewer), or you may plan for something more rigorous/systematic. —SS]

[If you have a supervisor for the project, you shouldn't just say they will read over the SRS. You should explain your structured approach to the review. Will you have a meeting? What will you present? What questions will you ask? Will you give them instructions for a task-based inspection? Will you use your issue tracker? —SS]

[Maybe create an SRS checklist? —SS]

3.3 Design Verification

[Plans for design verification —SS]

[The review will include reviews by your classmates —SS]

[Create a checklists? —SS]

3.4 Verification and Validation Plan Verification

[The verification and validation plan is an artifact that should also be verified. Techniques for this include review and mutation testing. —SS]

[The review will include reviews by your classmates —SS]

[Create a checklists? —SS]

3.5 Implementation Verification

[You should at least point to the tests listed in this document and the unit testing plan. —SS]

[In this section you would also give any details of any plans for static verification of the implementation. Potential techniques include code walkthroughs, code inspection, static analyzers, etc. —SS]

[The final class presentation in CAS 741 could be used as a code walkthrough. There is also a possibility of using the final presentation (in CAS741) for a partial usability survey. —SS]

3.6 Automated Testing and Verification Tools

[What tools are you using for automated testing. Likely a unit testing framework and maybe a profiling tool, like ValGrind. Other possible tools include a static analyzer, make, continuous integration tools, test coverage tools, etc. Explain your plans for summarizing code coverage metrics. Linters are another important class of tools. For the programming language you select, you should look at the available linters. There may also be tools that verify that coding standards have been respected, like flake9 for Python. —SS]

[If you have already done this in the development plan, you can point to that document. —SS]

[The details of this section will likely evolve as you get closer to the implementation. —SS]

3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

[Subsets of the tests may be in related, so this section is divided into different areas. If there are no identifiable subsets for the tests, this level of document structure can be removed. —SS]

[Include a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. —SS]

4.1.1 Area of Testing1

[It would be nice to have a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. If a section covers tests for input constraints, you should reference the data constraints table in the SRS. —SS]

Title for Test

1. test-id1

   Control: Manual versus Automatic

Initial State:

Input:

Output: [The expected result for the given inputs. Output is not how you are going to return the results of the test. The output is the expected result. —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

2. test-id2

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

4.1.2 Area of Testing2

...

4.2 Tests for Nonfunctional Requirements

The following tests assess the system-wide nonfunctional requirements. Accuracy measurements use a manually prepared answer key for a representative set of course outlines. Performance measurements will record the test environment, document characteristics, workload, and response times.

4.2.1 Course Outline Processing

1. **NFR-ACC-01: Extraction accuracy**

   Type: Dynamic, automatic

Initial State: The course-outline processing service is available and the representative test dataset and manually prepared answer key are stored.

Input/Condition: Submit each test outline for processing.

Output/Result: The final structured output contains the expected assessment dates, assessment weights, and learning outcomes with at least 95% accuracy for each field type. At least 95% of predefined missing, unclear, or incomplete fields are flagged for human review.

How test will be performed: Run the extraction pipeline on every outline, compare the confirmed structured output with the answer key, and calculate precision/recall or an equivalent field-level accuracy for each field type. Calculate missing-field recall as the proportion of known missing or incomplete fields that were flagged. Record the results and compare them with the targets in the Accuracy NFR.

2. **NFR-ACC-03: Policy and CEAB compliance accuracy**

   Type: Dynamic, automatic

   Initial State: The compliance-checking service is available and the representative test dataset has a manually prepared answer key identifying all predefined policy and CEAB compliance issues.

   Input/Condition: Submit each test outline for compliance checking.

   Output/Result: The system identifies at least 95% of the predefined policy and CEAB compliance issues and reports each identified issue in the verification results.

   How test will be performed: Run the compliance checker on every outline, compare the reported issues with the answer key, and calculate recall as the number of correctly identified predefined issues divided by the total number of predefined issues. Record false positives separately and compare the recall with the 95% target in the Accuracy NFR.

3. **NFR-PERF-01: Outline processing time**

   Type: Dynamic, automatic

Initial State: The processing service is available and the test environment has been recorded. A supported PDF course outline of no more than 30 pages is available.

Input/Condition: Submit a representative set of supported outlines, including documents with different layouts and extraction sizes.

Output/Result: At least 90% of successful processing runs complete within 60 seconds. Failures and delays caused by unavailable external dependencies are recorded separately.

How test will be performed: Run each outline through the complete document processing and information-extraction workflow, measure elapsed time from submission to structured output, and report the percentile and individual results against the 60-second target in the Performance NFR.

4.2.2 Calendar and Web Application

1. **NFR-PERF-02: Calendar response time**

   Type: Dynamic, automatic

   Initial State: The database contains the representative processed assessment dataset, the web application is running in the recorded test environment, and the user is authenticated.

   Input/Condition: Issue calendar-load requests using representative filters for course, year, and program under the expected concurrent workload.

   Output/Result: At least 95% of requests display the previously processed calendar data within 3 seconds.

   How test will be performed: Generate repeated calendar-load requests with the defined filters, measure the elapsed time until the calendar data is displayed, and report the response-time distribution and the percentage meeting the 3-second target in the Performance NFR.

2. **NFR-USE-01: Core task completion**

   Type: Dynamic, manual

Initial State: Representative CAS administrators and course instructors who were not involved in implementation and do not have programming or machine learning knowledge are provided with access to the deployed system and the task scenarios.

Input/Condition: Each participant attempts the designated core tasks: uploading an outline, reviewing extracted information, viewing and filtering the calendar, interpreting a flagged issue, and exporting available data, without programming or machine learning assistance.

Output/Result: At least 80% of participants complete each designated core task without assistance or technical knowledge. Any usability issues, errors, requests for help, or evidence of programming or machine learning assistance are recorded.

How test will be performed: Observe each participant using the system without providing task-specific or technical assistance, record task completion and time, and collect post-task feedback about ease of use. Calculate the completion rate for every task and verify that the results demonstrate use by non-technical participants. Compare the results with the Usability NFR.

3. **NFR-PORT-01: Hosting and browser portability**

Type: Dynamic, automatic and manual

Initial State: The system is deployed in the designated McMaster University hosting environment. The deployed web application and its core workflows are available on supported desktop operating systems in Google Chrome 120 or later, Mozilla Firefox 121 or later, and Safari 17 or later.

Input/Condition: Run the authentication, outline submission, information review, calendar viewing, and export workflows in each supported browser and on the designated hosting environment.

Output/Result: The system is usable in the designated McMaster University hosting environment and the core workflows complete successfully in each supported browser. Automated accessibility checks report no unresolved high-severity issues against the applicable AODA and WCAG 2.0 Level AA criteria.

How test will be performed: Run an automated accessibility scan, then manually complete the core workflows in every supported browser and on the designated hosting environment. Use keyboard navigation and a screen-reader check, and record the browser, operating system, hosting environment, applicable WCAG success criteria, results, and any deviations against the Portability and Accessibility NFRs.

4.2.3 Security, Reliability, and Deployment

1. **NFR-SEC-01: Access control and data protection**

   Type: Dynamic, automatic and manual

   Initial State: Test accounts exist for each supported role, including an unauthenticated user, a professor, and a CAS administrator. Test data includes records that each role should and should not be able to access.

   Input/Condition: Attempt to access protected pages and API endpoints without authentication, with the wrong role, and with valid permissions. Inspect deployment configuration and logs for exposed credentials or sensitive configuration values.

   Output/Result: Unauthenticated and unauthorized requests are rejected. Permitted users can perform only the actions assigned to their roles, and credentials and sensitive configuration values are not exposed.

   How test will be performed: Execute the access-control test cases for each role, verify both the user interface and direct endpoint responses, and inspect configuration handling. Record every unexpected access or exposure and compare the results with the Security NFR.

2. **NFR-REL-01: Invalid input and dependency failure recovery**

   Type: Dynamic, automatic

   Initial State: The database contains valid course, assessment, and calendar records. A submitted outline is retained by the system.

   Input/Condition: Submit an unsupported, malformed, oversized, incomplete, or unreadable outline, and separately make the extraction or recommendation service unavailable during processing.

Output/Result: The system reports an actionable error without crashing, corrupting or deleting existing records, and retains the submitted outline for retry. While the external service is unavailable, users can still view and edit stored information, view the calendar, and export available data.

How test will be performed: Run each invalid-input and dependency-failure scenario, verify the response and stored data before and after the failure, and retry processing after restoring the dependency. Record whether each predefined case meets the Reliability NFR.

3. **NFR-DEP-01: Deployment and maintainability**

Type: Static and dynamic, manual

Initial State: A clean designated McMaster University hosting environment and the documented installation and deployment instructions are available to a maintainer who did not perform the original setup.

Input/Condition: Deploy the system by following the documentation. Verify that the documentation identifies the required software, configuration values, environment variables, database setup, and service startup steps. Then implement representative isolated changes to document processing, scheduling, or compliance rules.

Output/Result: The maintainer can deploy the system to the designated McMaster University hosting environment and complete the core workflow without undocumented steps. The documentation contains all required setup information. The representative changes do not require unrelated components to be rewritten, and the effort for each change is recorded relative to the original implementation effort for the affected component.

How test will be performed: Give the documentation and clean hosting environment to a new maintainer without providing undocumented guidance. Observe the maintainer follow the installation and deployment instructions, execute the core workflow, and record any missing or ambiguous steps. Review the change diffs and effort records with the team, then document the results against the Installability, Deployability, and Maintainability NFRs.

4.3 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box

perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

References

Author Author. System requirements specification. https://github.com/..., 2019.

Canadian Engineering Accreditation Board. Accreditation Criteria and Procedures 2025. Engineers Canada, Ottawa, ON, Canada, November 2025. URL https://engineerscanada.ca/sites/default/files/2025-12/2025-Accreditation_Criteria_Procedures-EN.pdf. Accessed: Oct. 18, 2026.

McMaster University Senate. Undergraduate Course Management Policy. McMaster University, Hamilton, ON, Canada, May 2025. URL https://secretariat.mcmaster.ca/university-policies-procedures-guidelines/undergraduate-course-management-policy/undergraduate-course-management-policy-html/. Accessed: Oct. 18, 2026.

S. Mehta, A. Bornomala, H. S. Sagar, and V. Lai. Software Requirements Specification: CAS Course Deadline Management System. Team 19 (BVSH), Dept. of Computing and Software, McMaster University, Hamilton, ON, Canada, October 2026. URL https://github.com/shivyam/cas-course-optimization/blob/main/pdfs/SRS-IEEE/SRS.pdf. Accessed: Oct. 19, 2026.

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?